

SK네트웍스 Family AI과정 21기

모델링 및 평가 LLM 활용 소프트웨어

산출물 단계	모델링 및 평가
평가 산출물	LLM 활용 소프트웨어
제출 일자	2026.03.08.
깃허브 경로	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN21-FINAL-2Team.git
작성 팀원	전우영

목차

- 1. 프로젝트 개요
- 2. 시스템 개요
- 3. 시스템 구성요소
- 4. 코드 구조
- 5. 주요 코드
- 6. 프롬프트 최적화
- 7. 보안 고려사항
- 8. 테스트
- 9. 평가요소

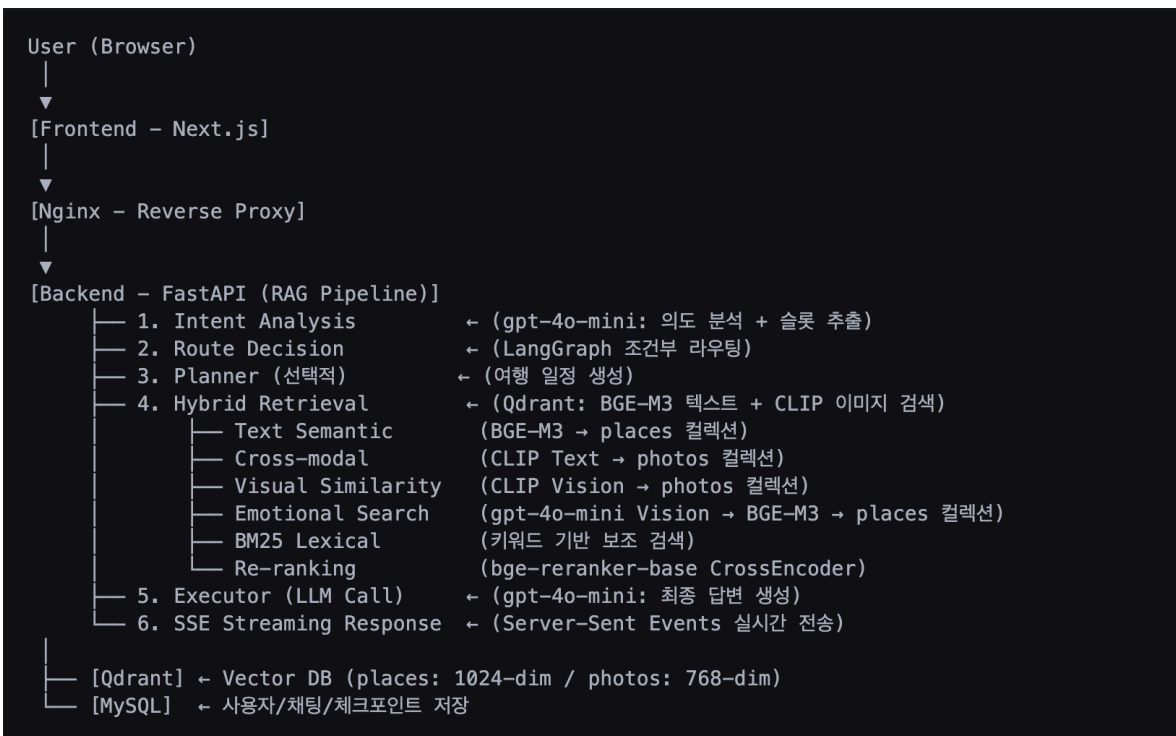
1. 프로젝트 개요

- Triver(트리버)는 외국인 관광객들의 국내 여행 전 과정을 책임지는 LLM 기반 AI 여행 에이전트.
- 소개인화 추천 엔진: 사용자의 명시적인 요청뿐만 아니라 이미지, 과거 취향, 실시간 상황을 복합적으로 분석하여 사용자에게 딱 맞는 장소를 추천
 - 장소 추천부터 코스 계획까지: 개별 장소 반환에서 나아가 사용자의 위치, 이동 수단 등을 종합적으로 분석하여 이동 효율을 극대화하는 알고리즘 기반 동선 설계
 - 여행 계획과 예약 관리: 여행지 검색부터 여행 상품 예약까지 가능한 채팅 인터페이스, 여러 플랫폼에서 예약한 다양한 바우처를 저장해둘 수 있는 나만의 여행 지갑 제공

2. 시스템 개요

- 시스템은 국내 여행지 추천 AI 챗봇으로, 벡터 데이터베이스(Qdrant)와 LLM(OpenAI gpt-4o-mini)을 연동하여 사용자 질문에 관광 데이터 기반 답변을 제공하는 RAG(Retrieval-Augmented Generation) 구조를 사용.
- LangGraph 기반의 멀티노드 에이전트 파이프라인을 통해 사용자 의도 분석 → 일정 계획 → 하이브리드 검색 → 응답 생성의 흐름으로 동작하며, 텍스트(BGE-M3)와 이미지(CLIP-ViT-L-14) 멀티모달 임베딩을 활용한 하이브리드 검색, SSE 기반 실시간 스트리밍 응답을 지원.
- LLM API Key 등 민감 정보는 .env 환경 변수로 관리되며, 프롬프트 최적화를 통해 빠른 응답 속도와 품질을 보장

3. 시스템 구성 요소



3.1. 주요 인프라 구성

서비스	이미지/빌드	포트	역할
backend	FastAPI (uvicorn)	8000	RAG 파이프라인 + API
frontend	Next.js	3000	사용자 인터페이스
nginx	Nginx	80	리버스 프록시
qdrant	qdrant/qdrant:latest	6333, 6334	벡터 데이터베이스
adminer	adminer	8080	DB 관리 UI

4. 코드 구조

```

backend/
├── main.py                # 서버 진입점
├── requirements.txt       # 의존성 패키지
├── Dockerfile            # 컨테이너 빌드
├── app/
│   ├── main.py           # FastAPI 앱 생성, 미들웨어, 라우터 등록
│   ├── agents/           # LangGraph 에이전트 파이프라인
│   │   ├── graph.py      # StateGraph 정의 (노드 등록, 엣지 연결)
│   │   ├── grapy_route.py # 조건부 라우팅 (route_by_intent, route_by_missing)
│   │   ├── intent.py     # 의도 분석 노드 (IntentType, 슬롯 추출)
│   │   ├── planner.py    # 여행 일정 생성 노드 (day/time_slot 구조)
│   │   ├── retriever.py  # 하이브리드 검색 노드 (PlaceRetriever 호출)
│   │   ├── executor.py   # 응답 생성 노드 (일반/추천/미싱 3종)
│   │   └── models/
│   │       ├── state.py  # TravelState TypedDict (그래프 상태 정의)
│   │       └── output.py # IntentOutput, IntentSlots Pydantic 모델
│   ├── api/
│   │   └── chat.py        # FastAPI 라우터 (엔드포인트)
│   │                       # 채팅 API (SSE 스트리밍, CRUD, 북마크)
│   ├── core/             # 앱 설정, 미들웨어
│   ├── database/         # DB 연결 관리
│   │   ├── connection.py # MySQL 커넥션 (SQLAlchemy, Singleton)
│   │   └── checkpoint.py # LangGraph 체크포인트 (AIOMySQLSaver)
│   ├── models/           # SQLAlchemy ORM 모델
│   │   └── chat.py        # ChatRoom, ChatMessage, ChatPlace
│   ├── retrieval/        # 검색 엔진
│   │   └── place.py       # PlaceRetriever (하이브리드 검색, RRF, BM25, Reranking)
│   ├── schemas/          # Pydantic 요청/응답 스키마
│   │   └── chat.py        # ChatMessageCreate, ChatRoomResponse 등
│   ├── services/         # 비즈니스 로직, 프롬프트
│   │   ├── prompts.py    # INTENT_PROMPT, PLANNER_PROMPT
│   │   ├── executor_prompt.py # EXECUTOR_PROMPT (추천/미싱/일반 3종)
│   │   ├── auto_start_prompt.py # 자동 시작 프롬프트
│   │   └── vision.py     # 이미지 → 감성 텍스트 변환 (gpt-4o-mini Vision)
│   ├── utils/            # 유틸리티
│   │   ├── config.py     # 모델/DB/검색 파라미터 설정
│   │   ├── llm_factory.py # LLM 싱글턴 팩토리 (ChatOpenAI, Tavily)
│   │   ├── llm_streaming.py # 토큰 스트리밍 (마크다운 버퍼링, ID 태그 제거)
│   │   ├── error_handler.py # 공통 예외 처리 (ErrorCode, AppException)
│   │   └── geocoder.py   # 좌표/주소 변환
│   ├── data/             # 환경 데이터 (JSONL/JSON)
│   │   ├── 12_관광지.jsonl
│   │   ├── 14_문화시설.jsonl
│   │   ├── 15_축제공연행사.jsonl
│   │   ├── 25_여행코스.jsonl
│   │   ├── 28_레포츠.jsonl
│   │   ├── 32_숙박.jsonl
│   │   ├── 39_음식점.jsonl
│   │   ├── 99_팝업스토어.json
│   │   └── visitkorea_data.json
│   ├── evaluation/       # RAG 평가 스크립트 (RAGAS 메트릭)
│   └── tests/            # 테스트 코드

```

5. 주요 코드

5.1. LangGraph 에이전트 파이프라인

그래프는 6개 노드로 구성되며, 조건부 라우팅으로 사용자 의도에 따라 분기합니다.

- 라우팅 로직(**grapy_route.py**)
 - **route_by_intent(): primary_intent** 값에 따라 분기
(TRIP_PLANNING → planner, GENERAL → executor_general, 나머지 → retriever)
 - **route_by_missing():** 필수 슬롯(위치, 기간 등)이 누락되면 executor_missing으로 분기하여 사용자에게 질문

```
# backend/app/agents/graph.py
def workflow():
    graph = StateGraph(TravelState)

    # 노드 등록
    graph.add_node("intent", intent_node)          # 의도 분석
    graph.add_node("planner", planner_node)        # 일정 생성
    graph.add_node("retriever", retriever_node)     # 하이브리드 검색
    graph.add_node("executor", executor_node)       # 최종 응답 (참소 추천)
    graph.add_node("executor_missing", executor_missing_node) # 추가 정보 요청
    graph.add_node("executor_general", executor_general_node) # 일반 대화

    # 라우팅
    graph.set_entry_point("intent")
    graph.add_conditional_edges("intent", route_by_intent) # TRIP_PLANNING → planner / GENERAL → executor_general / 그 외 → retriever
    graph.add_conditional_edges("planner", route_by_missing) # missing_slots 있으면 → executor_missing / 없으면 → retriever
    graph.add_edge("retriever", "executor")
    graph.add_edge("executor", END)
    graph.add_edge("executor_missing", END)
    graph.add_edge("executor_general", END)

    return graph
```

5.2. 예외처리 포함

- 공통 예외 처리 시스템 (**error_handler.py**)

```
# LLM 호출 시 예외 처리 (intent, planner, executor 공통 패턴)
try:
    result = await chain.invoke({...})
except Exception as e:
    return default_response # 오류 시 기본 응답 반환

# Tavily 웹 검색 폴백 (타임아웃 처리)
try:
    with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
        future = pool.submit(tavily.invoke, search_query)
        web_results = future.result(timeout=3.0)
except concurrent.futures.TimeoutError:
    print(f"Tavily fallback timeout after 3s")
except Exception as e:
    print(f"Tavily fallback failed: {e}")
```

- 에이전트 노드 예외 처리

```
# backend/app/utis/error_handler.py
class ErrorCode(IntEnum):
    TOKEN_EXPIRED = 1001 # 인증 토큰 만료
    TOKEN_INVALID = 1002 # 유효하지 않은 토큰
    USER_NOT_FOUND = 2001 # 사용자 미발견
    VALIDATION_ERROR = 3001 # 요청 검증 실패
    CHAT_ROOM_NOT_FOUND = 4001 # 채팅방 미발견
    INTERNAL_ERROR = 5001 # 서버 내부 오류

class AppException(Exception):
    def __init__(self, error_code: ErrorCode, message: str, status_code: int = 400):
        self.error_code = error_code
        self.message = message
        self.status_code = status_code
```

- 스트리밍 예외 처리

```
# SSE 스트리밍 중 예외 발생 시
except asyncio.CancelledError:
    db.rollback()
    raise
except Exception as e:
    traceback.print_exc()
    full_answer = "죄송합니다. 오류가 발생했습니다."
```

5.3. .env & 환경변수 설정

모든 민감 정보와 설정값은 .env 파일 및 환경 변수로 관리됩니다.

```
# LLM API
OPENAI_API_KEY=sk-...           # OpenAI API 키 (필수)
TAVILY_API_KEY=tvly-...         # Tavily 웹 검색 API 키 (폴백용)

# Vector DB (Qdrant)
QDRANT_HOST=qdrant              # Docker: qdrant / 로컬: localhost
QDRANT_PORT=6333

# Database (MySQL)
MYSQL_HOST=mysql                # Docker: mysql / 로컬: localhost
MYSQL_PORT=3306
MYSQL_USER=admin
MYSQL_PASSWORD=<password>
MYSQL_DATABASE=aichatbot

# 검색 파라미터
RETRIEVAL_PROFILE=serving       # serving(운영) 또는 evaluation(평가)
SERVING_RETRIEVER_CANDIDATE_K=20 # 후보 검색 수
SERVING_RETRIEVER_TOP_K=5       # 최종 반환 수
SERVING_RETRIEVER_RERANK_MAX_K=8 # 리랭킹 최대 수
```

5.4. LLM 모델 인스턴스 관리 (싱글턴 팩토리)

Temperature마다 인스턴스 생성하여 단일 로드로 모델 관리

```
# backend/app/utils/llm_factory.py
class LLMFactory:
    _llm_instances: dict[tuple[str, float], ChatOpenAI] = {}

    @classmethod
    def get_llm(cls, model: str = "gpt-4o-mini", temperature: float = 0):
        key = (model, float(temperature))
        if key not in cls._llm_instances:
            cls._llm_instances[key] = ChatOpenAI(model=model, temperature=temperature)
        return cls._llm_instances[key]

    @classmethod
    def get_tavily(cls, max_result=3):
        # 검색 결과 없을 때 웹 검색 폴백
        ...
```

6. 프롬프트 최적화

6.1. 역할 기반 프롬프트 분리

각 노드별 목적에 맞는 전용 프롬프트를 사용하여 LLM 호출 효율 극대화

프롬프트	파일	용도
INTENT_PROMPT	services/prompts.py	의도 분석 + 슬롯 추출 (Structured Output)
PLANNER_PROMPT	services/prompts.py	여행 일정 생성 (day/time_slot)
EXECUTOR_PROMPT	services/executor_prompt.py	장소 추천 + 네이버 지도 링크 포함 응답
EXECUTOR_MISSING_INFO_PROMPT	services/executor_prompt.py	부족 정보 질문 생성
EXECUTOR_GENERAL_PROMPT	services/executor_prompt.py	일반 대화 응답
IMAGE_TO_EMOTIONAL_PROMPT	services/vision.py	이미지 → 감성 텍스트 변환

6.2. 주요 최적화 전략

- **Structured Output**

Intent 분석 시 IntentOutput Pydantic 모델을 사용하여 LLM이 정해진 스키마로만 응답하도록 강제 → 파싱 오류 방지, 후처리 불필요

- **Context 길이 제한**

검색 결과를 top_k(기본 5개)로 제한하여 프롬프트에 포함되는 문서 수를 최소화

- **temperature 분리**

의도 분석(0.0 - 정확도 우선) / 응답 생성(0.3~0.7 - 자연스러움) 등 목적별 temperature 조절

- **RRF(Reciprocal Rank Fusion)**

다채널 검색 결과를 랭크 기반으로 통합하여 중복 제거 및 다양성 확보

- **BM25 조건부 활성화**

시맨틱 검색 점수가 낮을 때만(score_threshold < 0.22) BM25 키워드 검색을 보조로 활성화

6.3. 멀티모달 검색 흐름

```

[텍스트 입력] → BGE-M3 인코딩 → places 컬렉션 시맨틱 검색 (가중치 1.0)
                ↳ CLIP Text 인코딩 → photos 컬렉션 크로스모달 검색 (가중치 0.5)

[이미지 입력] → CLIP Vision 인코딩 → photos 컬렉션 시각 유사도 검색 (가중치 1.0)
                ↳ gpt-4o-mini Vision → 감성 텍스트 → BGE-M3 인코딩 → places 컬렉션 (가중치 0.8)

[RRF 결합] → [CrossEncoder 리랭킹 (bge-reranker-base)] → 최종 후보 반환
  
```

7. 보안 고려사항

프롬프트	파일
API Key 보호	모든 API 키(<code>OPENAI_API_KEY</code> , <code>TAVILY_API_KEY</code>)는 <code>.env</code> 파일 또는 시스템 환경 변수에서 로드하며, 코드에 하드코딩하지 않음
<code>.gitignore</code> 등록	<code>.env</code> 파일은 <code>.gitignore</code> 에 등록하여 원격 저장소에 커밋되지 않게 함
Docker 환경 변수 분리	<code>docker-compose.yml</code> 의 <code>env_file</code> 과 <code>environment</code> 키를 통해 환경 변수 컨테이너별 주입
DB 커넥션 보안	<code>pool_pre_ping=True</code> , <code>pool_recycle=3600</code> 설정으로 커넥션 상태를 지속적으로 검증하며, 비밀번호는 환경 변수로 관리
인증 체계	JWT 토큰 기반 인증을 적용하며, <code>ErrorCode.TOKEN_EXPIRED(1001)</code> , <code>TOKEN_INVALID(1002)</code> 등 세분화된 에러 코드로 인증 실패 처리
입력 검증	FastAPI의 Pydantic 스키마를 통해 모든 API 입력을 자동 검증하고, <code>RequestValidationError</code> 를 공통 JSON 응답으로 변환

8. 테스트

8.1. 테스트 시나리오

유형	질문	답변
장소 추천 질문	“홍대 근처 분위기 좋은 카페 추천해줘”	홍대 근처에서 분위기 좋은 카페를 찾고 계시군요! 😊 연남동 카페거리에 있는 '엔트러사이트 연남'은 넓은 공간과 빈티지한 인테리어로 유명해요. 오래된 공장을 개조한 독특한 분위기가 매력적이고, 창가 자리에서 여유롭게 시간을 보내기 좋습니다. 엔트러사이트 연남

여행 일정 생성	"홍대 2박 3일 여행 일정 짜줘"	부족한 정보(인원, 예산, 관심사 등)를 먼저 질문 → 정보 수집 후 day/time_slot 기반 일정 생성
이미지 기반 추천	전광판 가득한 도심 사진 전송 + "이런 느낌의 장소 추천해줘"	CLIP Vision으로 시각 유사도 검색 + GPT-4o-mini Vision으로 감성 텍스트 추출 → 분위기가 유사한 해안 명소 추천
일반 대화	"안녕 오늘 날씨 어때?"	일반 대화로 라우팅 → 친근한 답변 + 여행 관련 자연스러운 유도

8.2. 테스트 커버리지

테스트 파일	검증 내용
<code>test_graph_routing.py</code>	LangGraph 조건부 라우팅 정확성
<code>test_retriever_selection.py</code>	검색 후보 선택 및 다양성
<code>test_retrieval_profile_config.py</code>	serving/evaluation 파라미터 설정
<code>test_llm_streaming.py</code>	토큰 스트리밍, 마크다운 링크 버퍼링
<code>test_place_rerank_fallback.py</code>	리랭커 폴백 동작
<code>test_chat_stream.py</code>	SSE 스트리밍 엔드투엔드
<code>test_evaluation_*.py</code>	RAGAS 기반 검색/생성/추천 품질 평가

9. 평가요소표

평가 항목	대응 내용
벡터 DB와 LLM이 목적에 맞는 프롬프트로 효율적 연동	Qdrant 하이브리드 검색(BGE-M3 + CLIP) 결과를 <code>EXECUTOR_PROMPT</code> 에 Context로 주입하여 장소 데이터 기반 답변 생성. RRF + CrossEncoder 리랭킹으로 검색 품질 최적화
예상치 못한 상황 예외 처리 포함	LLM 호출 <code>try/except</code> 처리, Tavily 폴백 3초 타임아웃, SSE 스트리밍 <code>CancelledError</code> 롤백, <code>AppException</code> 공통 예외 체계 (ErrorCode 1001~5001)
코드 모듈화 및 주석 작성	LangGraph 노드별 파일 분리 (<code>intent.py</code> , <code>planner.py</code> , <code>retriever.py</code> , <code>executor.py</code>), 유틸/서비스/모델/스키마 계층 분리, 각 모듈에 한국어 주석 포함

보안 정보 노출 방지	.env 파일 + 환경 변수 활용, .gitignore 등록, Docker env_file 분리, JWT 토큰 인증, Pydantic 입력 검증
빠른 응답을 위한 프롬프트 최적화	Structured Output(IntentOutput), top_k 제한(기본 5개), LLM 싱글턴 캐싱, 단계별 프롬프트 분리, temperature 목적별 조절, 조건부 BM25 활성화